

Chapter 45.

Data Ingestion & Integration

Full, incremental, hash-based and change-data-capture loads; reliable file loading; schema changes; API ingestion; build versus buy; and the limits of web data.

Draft chapter

Contents

Chapter 45. Data Ingestion & Integration

Why this matters

In plain English

45.1 Where company data comes from

45.2 Setting up the practice environment

45.3 Full loads

45.4 Incremental loads, and the update they miss

45.5 Detecting change with hashes

45.6 Change data capture

45.7 Loading files reliably

45.8 When the source changes shape

45.9 Pulling data from APIs

45.10 Build or buy: connectors versus custom code

45.11 Web data: legal and ethical limits

Common mistakes and how to spot them

In the real world: the orders that never cancelled

Tools

The project: incremental ingestion for Riverstone

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 45. Data Ingestion & Integration

Part V — Data Engineering, Integration & Scale

Chapter at a glance

You will learn to: name the kinds of source systems a data platform ingests from, and what makes each one hard · choose between full loads, incremental loads, and change data capture, and explain what each one can miss · show, with a real test, why a "new rows only" load silently loses updates · detect inserted, changed, and deleted rows with hashes, and skip files that haven't changed · read real change events from PostgreSQL's log · load messy CSV files without silent type errors, and reconcile what you loaded against the file · recognize schema changes before they break a load · pull data from a paginated, authenticated, rate-limited API with retries · decide when to build a connector and when to buy one · judge the legal and ethical limits of collecting web data.

Before you start: Chapter 2 (formats, hashes, and APIs), Chapters 12–13 (SQL), Chapter 18 (Python for analysis), and Chapter 29 (Python as software). Chapter 32 (dbt) helps but isn't required.

Time needed: 14–18 hours of reading and practice, spread over two to three weeks.

Tools: PostgreSQL 16, Python 3.12 with `psycopg2`, `duckdb`, and `requests`, and the Chapter 45 companion folder. Everything runs on your own computer.

Practice data: `riverstone_source`, a private copy of the Riverstone one-year database that plays the part of the ERP and the CRM; two dispatch files from the Bhiwandi Main warehouse; and a small local API that stands in for the CRM. Every output shown is from a real run.

Why this matters

Every report, dashboard, model, and automation you've built so far started with data that was already sitting in a table. Somebody put it there. In most companies, that somebody is a data engineer, and the first job is **ingestion**: getting data out of the systems where the business runs and into a place built for analysis, correctly, every day, without anyone copying files by hand.

Ingestion looks simple and isn't. The ERP doesn't tell you which orders changed yesterday. The warehouse's dispatch file arrives with dates in a different format and quantities written as text. The CRM's API only returns 20 records at a time, and refuses your requests if you ask too quickly. A source system adds a column on a Tuesday and your load breaks on Wednesday, or worse, keeps running with the wrong numbers.

The failures that hurt most in ingestion aren't crashes. They're loads that succeed and quietly deliver the wrong data. This chapter shows three of them happening on Riverstone data, and how to catch each one. Get this layer right and everything built on top of it can be trusted. Get it wrong and every chart in the company is wrong in ways nobody notices.

In plain English

Think about how a large kitchen gets its ingredients.

- Ingredients arrive from many **suppliers**: a vegetable vendor, a dairy, a spice wholesaler. Each delivers differently: some bring crates, some send a list and a truck, some you must collect. Those are **source systems**, each with its own way of handing over data.
- On the first day, the kitchen takes **everything** each supplier has. That's a **full load**.
- After that, it only wants what's **new or different** since yesterday. That's an **incremental load**. But if a supplier quietly swaps yesterday's milk for a different brand and keeps the same crate number, a kitchen that only counts new crates will never notice. That's the update a naive incremental load misses.
- A careful kitchen compares a **fingerprint** of each item against yesterday's: same weight, same label, same batch code. Any difference is a change. That's **change detection with hashes**.
- The best suppliers send a **delivery log** listing every change as it happens: "added two crates, replaced one". That's **change data capture**.
- At the receiving door, someone checks each delivery against the delivery note, sets aside anything damaged, and writes down what arrived. That's **reconciliation** and a **load log**.
- One supplier changes its packaging without warning. If the receiving clerk only checks crate counts, the new packaging sails through. That's a **schema change**.

Ingestion is the receiving door of the data platform. This chapter is about running it well.

45.1 Where company data comes from

Before you design any load, know what kind of source you're dealing with. Each kind fails in its own way.

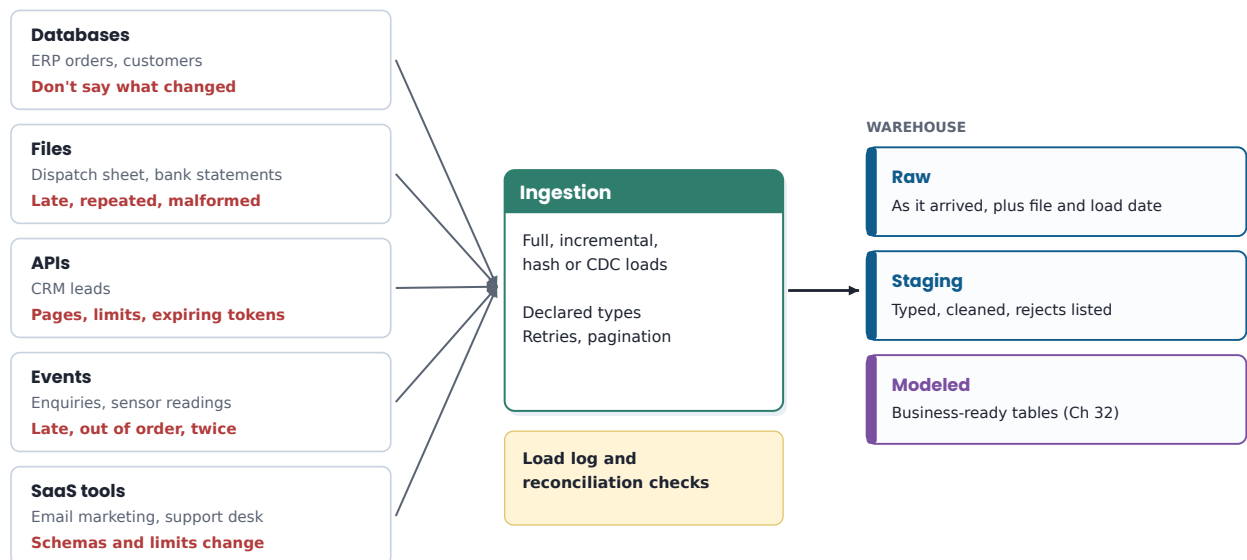


Figure 45.1 – The main kinds of source, and the difficulty each brings. Ingestion writes to a raw layer first, so the original data is kept even when later steps change.

Source type	Riverstone example	How you usually read it	What makes it hard
Operational databases	The ERP's orders, order lines, customers	SQL queries, or the database's change log	They're built to run the business, not to tell you what changed; heavy queries can slow the business down
Files	Bhiwandi Main's daily dispatch sheet; bank statements	Reading CSV, Excel, JSON, or Parquet files from a folder, SFTP server, or cloud storage	Late, duplicated, renamed, or malformed files; formats chosen by a person, not a system
APIs	The CRM's leads	HTTP requests with authentication (Chapter 2)	Pagination, rate limits, expiring tokens, changing versions
Events	Website enquiries; plant sensor readings (Chapter 50)	Streams and message queues	Huge volumes; events arrive late, out of order, or twice
SaaS tools	Email marketing, the support desk	Their APIs, usually through a connector	Limits and schemas you don't control, and change without notice

Two terms appear throughout this part of the book.

- A **source system** is any system where data is first recorded: the ERP, the CRM, a spreadsheet the warehouse maintains.
- A **system of record** is the source that's officially correct for a piece of data. Riverstone's ERP is the system of record for orders; the CRM is the system of record for leads. When two systems disagree, the system of record wins. Chapter 51 relies on this when data flows back the other way.

Layers in the warehouse

Ingested data lands in layers, each with a purpose.

- The **raw** layer holds data as it arrived, with as little change as possible. If a later step is wrong, you can rebuild from raw without asking the source again.

- The **staging** layer holds the same data cleaned and typed: dates as dates, quantities as numbers, consistent names.
- The **modeled** layer holds the business-ready tables that reports use, often built with dbt (Chapter 32).

This chapter covers raw and staging. Chapter 49 covers how the warehouse stores these layers, and Chapter 32 covers modeling on top of them.

Simplification note. The chapter uses a local DuckDB file as the warehouse, so everything runs on your computer for free. The patterns are the same on cloud warehouses such as Snowflake, BigQuery, Redshift, or Databricks; Chapter 49 compares them.

45.2 Setting up the practice environment

The companion folder `companion/ch45/` contains four small helper files. You'll use them, but you won't need to change them.

File	What it does
<code>reset_ch45.py</code>	Creates <code>riverstone_source</code> , a private copy of <code>riverstone_2025</code> , deletes the local warehouse, and writes the practice files. Run it any time to start again.
<code>apply_day.py</code>	Simulates later business days in the ERP: new orders, and status changes to existing ones.
<code>mock_crm_api.py</code>	A tiny local web server that behaves like the CRM's API, including authentication, pagination, and rate limiting.
<code>make_files.py</code>	Writes Bhiwandi Main's dispatch files, messy in realistic ways.

Your copy of `riverstone_source` is disposable. The shared practice databases from Chapter 12 are never changed.

Change detection in section 45.5 needs one PostgreSQL setting: `wal_level = logical`. It's covered there, with the exact steps.

Start Python in the `companion/ch45` folder and run:

```
import os, hashlib, time, json
import psycopg2, duckdb, requests
from reset_ch45 import reset
from apply_day import apply_day

reset()
SRC = "dbname=riverstone_source"
wh = duckdb.connect("warehouse/riverstone_wh.duckdb")
print("source and warehouse ready")
```

```
source and warehouse ready
```

How it works.

- `reset()` builds a fresh environment. `SRC` is the connection string for the source database; if your PostgreSQL needs a user name or password, add them, for example `"dbname=riverstone_source user=postgres password=..."`.
- `duckdb.connect()` opens (or creates) the warehouse as a single file. **DuckDB** is an analytical database that runs inside your Python process, with no server to install.

Tool note. The outputs in this chapter were produced with Python 3.12.3, DuckDB 1.5.5, psycopg2 2.9.13, requests 2.33.1, and PostgreSQL 16. Newer versions may format a few outputs slightly differently.

45.3 Full loads

A **full load** copies everything from a source table into the warehouse, replacing what was there. It's the simplest pattern, and often the right one.

```
def fetch_rows(sql):
    conn = psycopg2.connect(SRC)
    with conn, conn.cursor() as cur:
        cur.execute(sql)
        rows = cur.fetchall()
        cols = [d[0] for d in cur.description]
    conn.close()
    return cols, rows

cols, rows = fetch_rows("SELECT order_id, customer_id, order_date, status, sales_rep_id FROM
orders")

wh.execute("CREATE SCHEMA IF NOT EXISTS raw")
wh.execute("""
    CREATE OR REPLACE TABLE raw.orders (
        order_id      INTEGER PRIMARY KEY,
        customer_id   INTEGER NOT NULL,
        order_date     DATE NOT NULL,
        status         VARCHAR NOT NULL,
        sales_rep_id   INTEGER
    )""")
wh.executemany("INSERT INTO raw.orders VALUES (?, ?, ?, ?, ?)", rows)

print("rows read from source :", len(rows))
print("rows in raw.orders    :", wh.execute("SELECT COUNT(*) FROM raw.orders").fetchone()[0])
```

```
rows read from source : 175
rows in raw.orders    : 175
```

How it works, line by line.

- `fetch_rows()` opens a connection to the source, runs one query, and returns the column names and all rows. The `with conn` block commits or rolls back automatically.
- `CREATE OR REPLACE TABLE` throws away yesterday's copy and creates an empty table with explicit types. Declaring types, instead of letting the tool guess, is the first habit of reliable ingestion.

- `executemany()` inserts every row. The `?` placeholders keep values separate from the SQL text, which avoids quoting mistakes and SQL injection.
- The last two lines compare the count read with the count stored. Chapter 12 asked you to run `COUNT(*)` against the source after loading; this is the same check, automated.

Reconcile more than the row count

Row counts can match while the contents don't. A slightly stronger check compares counts by a meaningful category:

```
src_status = dict(fetch_rows("SELECT status, COUNT(*) FROM orders GROUP BY status")[1])
wh_status = dict(wh.execute("SELECT status, COUNT(*) FROM raw.orders GROUP BY status").fetchall())
for s in sorted(src_status):
    print(f"{s:<10} source={src_status[s]:>3} warehouse={wh_status.get(s, 0):>3}")
```

```
Cancelled source= 2 warehouse= 2
Delivered source=171 warehouse=171
Pending   source= 1 warehouse= 1
Shipped   source= 1 warehouse= 1
```

Everything matches: 175 orders, 171 of them delivered. A **reconciliation check** like this compares a summary of the source with the same summary of the warehouse. It's cheap, and it catches a surprising number of problems.

When a full load is the right choice

Full loads are **simple, self-correcting** (every run fixes any earlier mistake), and they **capture updates and deletes** automatically, because the whole table is replaced. Choose them when:

- the table is small or medium: Riverstone's customers, products, and employees are ideal;
- the source can handle reading the whole table without slowing the business down;
- the load finishes comfortably inside the time you have.

They stop being practical when a table is large, changes slowly, or sits in a busy production system. Reading a hundred million order lines every night to find the few thousand that changed is slow, expensive, and hard on the ERP. That's where incremental loads come in.

45.4 Incremental loads, and the update they miss

An **incremental load** reads only the rows that are new or changed since the last load. To know what "since the last load" means, it keeps a **watermark**: a stored value marking how far the previous load got, such as the highest ID or the latest timestamp it saw.

It's now 2 January 2026. The ERP has had a business day. Let's apply it, then run the most common incremental pattern: load rows with an ID above the watermark.

```

apply_day(1)

watermark = wh.execute("SELECT MAX(order_id) FROM raw.orders").fetchone()[0]
cols, new_rows = fetch_rows(f"""
    SELECT order_id, customer_id, order_date, status, sales_rep_id
    FROM orders
    WHERE order_id > {watermark}
    ORDER BY order_id""")
wh.executemany("INSERT INTO raw.orders VALUES (?, ?, ?, ?, ?)", new_rows)

print("watermark before load:", watermark)
print("new rows loaded      :", [r[0] for r in new_rows])

```

```

watermark before load: 10175
new rows loaded      : [10176, 10177]

```

The load ran, found two new orders, and finished without errors. Now run the reconciliation check from section 45.3 again:

```

src_status = dict(fetch_rows("SELECT status, COUNT(*) FROM orders GROUP BY status")[1])
wh_status = dict(wh.execute("SELECT status, COUNT(*) FROM raw.orders GROUP BY status").fetchall())
for s in sorted(src_status):
    flag = "" if src_status[s] == wh_status.get(s, 0) else " <-- differs"
    print(f"{s:<10} source={src_status[s]:>3} warehouse={wh_status.get(s, 0):>3}{flag}")
print(wh.execute("SELECT order_id, status FROM raw.orders WHERE order_id IN (10174, 10175) ORDER BY
order_id").fetchall())

```

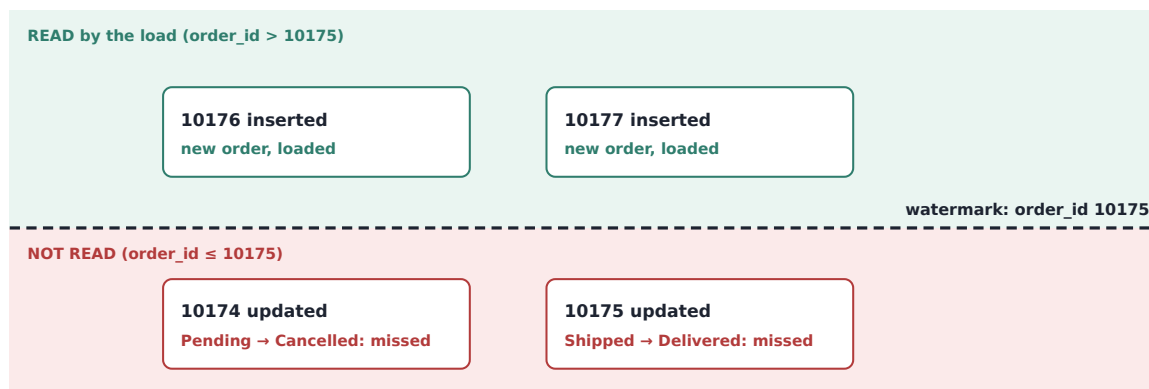
```

Cancelled source= 3 warehouse= 2 <-- differs
Delivered source=172 warehouse=171 <-- differs
Pending   source= 2 warehouse= 3 <-- differs
[(10174, 'Pending'), (10175, 'Shipped')]

```

Reading it. The row counts match (177 in both), but three status counts don't. On 2 January, the ERP also *updated* two existing orders: order 10174 was cancelled, and order 10175 was delivered. Their IDs are below the watermark, so the load never looked at them. The warehouse still says 10174 is Pending and 10175 is Shipped. A dashboard built on it would count a cancelled order as open business.

- 1 JANUARY: warehouse loaded up to order 10175
2 JANUARY: what happens in the ERP, and what an ID-watermark load reads



The load finished without errors. Only a reconciliation against the source showed the two stale orders.

Figure 45.2 – Why an ID watermark misses updates. The load only looks above the line; changes to older rows happen below it.

Watch out: a successful load is not a correct load. The watermark load above raised no error and loaded exactly the rows it was designed to load. Only a reconciliation check against the source revealed the problem. Never judge a pipeline by whether it finished.

Better watermarks, and their limits

The usual fix is a **last-updated timestamp**: a column such as `updated_at` that the source sets every time a row changes. Then the load reads `WHERE updated_at > watermark`, and updates are caught.

Riverstone's `orders` table, like many real ERP tables, has no such column. And where one exists, check three things before trusting it:

- **Does every change update it?** Bulk fixes run directly in the database often skip it.
- **Is it set when the change is committed, or when it started?** A long transaction that began before your last load can commit a timestamp you've already passed.
- **What about deletes?** A deleted row has no timestamp to find. Incremental loads by timestamp never see deletes.

A common safety margin is to **look back**: load rows where `updated_at > watermark - interval '1 hour'` and let the upsert (section 45.5) absorb the duplicates. When no reliable column exists, you need a different way to detect change, which is the next section.

45.5 Detecting change with hashes

Chapter 2 introduced **hashes**: short fingerprints computed from data, where any change to the data gives a completely different fingerprint. Chapter 2 also said pipelines use them to tell whether a file has changed since yesterday. Here's that idea applied to rows, and then to files.

Row hashes

Compute a hash of each row's values in the source, and the same hash in the warehouse. Rows whose hashes differ have changed.

```
HASH_SQL = """
    SELECT order_id,
           md5(concat_ws('|', order_id, customer_id, order_date, status, sales_rep_id)) AS row_hash
    FROM orders"""
source_hashes = dict(fetch_rows(HASH_SQL)[1])

wh_hashes = dict(wh.execute("""
    SELECT order_id,
           md5(concat_ws('|', order_id, customer_id, strftime(order_date, '%Y-%m-%d'), status,
sales_rep_id))
    FROM raw.orders""").fetchall())

inserted = sorted(set(source_hashes) - set(wh_hashes))
deleted = sorted(set(wh_hashes) - set(source_hashes))
changed = sorted(k for k in set(source_hashes) & set(wh_hashes) if source_hashes[k] !=
wh_hashes[k])
print("inserted:", inserted)
print("deleted :", deleted)
print("changed :", changed)
```

```
inserted: []
deleted : []
changed : [10174, 10175]
```

How it works.

- `concat_ws('|', ...)` joins a row's values into one string with `|` between them, for example `10174|9|2025-12-21|Pending|5`. The separator matters: without it, the values `1` and `23` would join to the same `123` as `12` and `3`.
- `md5()` turns that string into a 32-character fingerprint. MD5 is fine for detecting change; it isn't safe for security, which Chapter 2 explained.
- The warehouse side uses `strftime` to format the date exactly as PostgreSQL does. **Both sides must build the string identically**, or every row will look changed.
- Comparing the two dictionaries gives three lists. A key only in the source was inserted; a key only in the warehouse was deleted; a key in both with different hashes was changed.

Reading it. The hash comparison finds exactly the two orders the watermark load missed, 10174 and 10175, and confirms there were no deletes. (The two new orders were already loaded, so they're not "inserted" here.)

Applying the changes with an upsert

An **upsert** inserts a row if its key is new, and updates it if the key already exists. In DuckDB it's written `INSERT OR REPLACE`; PostgreSQL uses `INSERT ... ON CONFLICT ... DO UPDATE`, and many warehouses use `MERGE`.

```
ids = ", ".join(str(i) for i in changed)
cols, fresh = fetch_rows(f"""
    SELECT order_id, customer_id, order_date, status, sales_rep_id
    FROM orders WHERE order_id IN ({ids})""")
wh.executemany("INSERT OR REPLACE INTO raw.orders VALUES (?, ?, ?, ?, ?)", fresh)
print(wh.execute("SELECT order_id, status FROM raw.orders WHERE order_id IN (10174, 10175) ORDER BY
order_id").fetchall())
```

```
[(10174, 'Cancelled'), (10175, 'Delivered')]
```

Both orders are now correct.

Why upserts make loads safe to repeat

A load that can be run twice with the same result is **idempotent**. It matters because loads fail halfway and get re-run, schedulers retry, and people press "run" again. A plain `INSERT` isn't idempotent; watch what happens if yesterday's two new orders are inserted a second time:

```
try:
    wh.executemany("INSERT INTO raw.orders VALUES (?, ?, ?, ?, ?)", new_rows)
except duckdb.ConstraintException as e:
    print("plain INSERT, second run:", str(e).splitlines()[0][:80])

wh.executemany("INSERT OR REPLACE INTO raw.orders VALUES (?, ?, ?, ?, ?)", new_rows)
wh.executemany("INSERT OR REPLACE INTO raw.orders VALUES (?, ?, ?, ?, ?)", new_rows)
print("upsert, run twice    :", wh.execute("SELECT COUNT(*) FROM raw.orders").fetchone()[0], "rows")
```

```
plain INSERT, second run: Constraint Error: Duplicate key "order_id: 10176" violates primary key
constraint
upsert, run twice      : 177 rows
```

Here the primary key stopped the duplicate, which is exactly why raw tables should have one. Without a primary key, the plain insert would have silently doubled two orders. The upsert can run any number of times and leave 177 rows. Chapter 46 builds whole pipelines on this property, and tests it by making a run fail on purpose.

The cost of row hashing

Hash comparison catches inserts, updates, and deletes without any help from the source. The price is that it reads **every row's key and hash** on every run. For Riverstone's 177 orders that's nothing; for a table of hundreds of millions of rows, it's a heavy scan of a production database each night. Common compromises:

- hash only a recent window, such as the last 90 days, where changes usually happen, plus a full comparison once a week;
- hash in chunks (by month or ID range), compare chunk hashes first, and drill into rows only where a chunk differs;
- use change data capture instead (section 45.6), which doesn't need to scan at all.

File hashes: skipping files that haven't changed

The same idea applies to whole files. Bhiwandi Main saves its dispatch sheet into a shared folder every evening. Sometimes someone saves it twice; sometimes the file is re-sent unchanged. A file hash tells you whether you've already loaded exactly this content.

```
def file_sha256(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(65536), b''):
            h.update(chunk)
    return h.hexdigest()

wh.execute("""
CREATE TABLE IF NOT EXISTS raw.file_log (
    file_name VARCHAR, sha256 VARCHAR PRIMARY KEY, loaded_for DATE)""")

def should_load(path, loaded_for):
    digest = file_sha256(path)
    seen = wh.execute("SELECT file_name FROM raw.file_log WHERE sha256 = ?", [digest]).fetchone()
    if seen:
        return f"skip: same content as {seen[0]}"
    wh.execute("INSERT INTO raw.file_log VALUES (?, ?, ?)", [os.path.basename(path), digest,
loaded_for])
    return "load"

print(should_load("exports/dispatch_2026-01-02.csv", "2026-01-02"))
print(should_load("exports/dispatch_2026-01-02.csv", "2026-01-02"))
```

```
load
skip: same content as dispatch_2026-01-02.csv
```

How it works. `file_sha256()` reads the file in 64 KB chunks, so it works on files far larger than memory. `should_load()` records each new file's hash in a **load log** table; a second file with the same hash is skipped, even if it has a different name. A file that differs by a single byte gets a different hash and is loaded.

Watch out: the same content isn't always the same file. A hash tells you the *content* is identical. If a warehouse legitimately sends two identical dispatch files for two different days (no dispatches on both), skipping the second would be wrong. Store the date or batch the file represents alongside the hash, and decide explicitly what "already loaded" means for each source.

45.6 Change data capture

Every change to a PostgreSQL database is first written to a **write-ahead log** (WAL): a running record of every insert, update, and delete, used for crash recovery and replication. **Change data capture** (CDC) reads changes from that log instead of querying tables. It sees every insert, update, and delete, in commit order, without scanning anything.

	Full load	Incremental (watermark)	Hash comparison	Change data capture
Reads	Whole table	Rows past last ID/time	All keys and hashes	The change log
Inserts	Yes	Yes	Yes	Yes
Updates	Yes	Only with reliable <code>updated_at</code>	Yes	Yes, in order
Deletes	Yes	No	Yes	Yes
Load on source	Heavy for big tables	Light	Medium to heavy	Very light
Complexity	Simplest	Simple, can miss silently	Moderate	Settings, permissions, slots

Figure 45.3 – Four ways to keep a warehouse table in step with its source. Notice which ones can see deletes.

Turning CDC on

CDC needs PostgreSQL to write enough detail into the log. That's controlled by one setting, which needs a server restart:

1. Find your configuration file: in `psql`, run `SHOW config_file;`
2. Set `wal_level = logical` in that file (the default is `replica`).
3. Restart PostgreSQL, and check with `SHOW wal_level;`

On a company database, this is a change for the database administrator, not for you: it increases log volume, and it needs a user with replication permission. Managed cloud databases expose the same setting under a different name; check the provider's documentation.

Simplification note. Production CDC usually runs through a dedicated tool, such as the open-source Debezium, or through a managed connector, and it streams changes continuously (Chapter 50). This section uses PostgreSQL's built-in `test_decoding` plugin, which prints changes as readable text. It's meant for learning and testing, not for production pipelines.

Reading real changes

A **replication slot** is a bookmark in the log. PostgreSQL keeps every change after the bookmark until a reader consumes it, so nothing is lost between reads. Create a slot, then let the ERP have another business day (5 January 2026):

```
conn = psycopg2.connect(SRC); conn.autocommit = True
cur = conn.cursor()
cur.execute("SELECT slot_name FROM pg_create_logical_replication_slot('ch45_slot',
'test_decoding')")
print("slot created:", cur.fetchone()[0])

apply_day(2)

cur.execute("SELECT data FROM pg_logical_slot_get_changes('ch45_slot', NULL, NULL)")
for (line,) in cur.fetchall():
    if line.startswith("table "):
        print(line)
```

```
slot created: ch45_slot
table public.orders: UPDATE: order_id[integer]:10176 customer_id[integer]:9 order_date[date]:'2026-01-02' status[character varying]:'Shipped' sales_rep_id[integer]:5
table public.orders: INSERT: order_id[integer]:10178 customer_id[integer]:17
order_date[date]:'2026-01-05' status[character varying]:'Pending' sales_rep_id[integer]:3
table public.order_items: INSERT: order_item_id[integer]:334 order_id[integer]:10178
product_id[integer]:102 quantity[integer]:25 unit_price[numeric]:750.00 discount_pct[numeric]:0.00
```

How it works.

- `pg_create_logical_replication_slot()` creates the bookmark and names the plugin that turns log records into text.
- `apply_day(2)` runs the day's changes in the ERP, in one transaction.
- `pg_logical_slot_get_changes()` returns every change since the bookmark, then moves the bookmark forward. The plugin also returns `BEGIN` and `COMMIT` lines with transaction numbers, which change on every run, so the loop prints only the table changes.

Reading it. Three changes, in the order they were committed: order 10176 moved to Shipped, a new order 10178 was created, and its order line was inserted. Each line carries the full new row, so the warehouse can apply it with an upsert. No table was scanned.

Read the slot again, and there's nothing waiting:

```
cur.execute("SELECT data FROM pg_logical_slot_get_changes('ch45_slot', NULL, NULL)")
print("changes waiting:", len(cur.fetchall()))
```

```
changes waiting: 0
```

Both methods agree

As a check, compare the hash method from section 45.5 against the same day's changes. It should find what CDC found.

```
source_hashes = dict(fetch_rows(HASH_SQL)[1])
wh_hashes = dict(wh.execute("""
    SELECT order_id,
           md5(concat_ws('|', order_id, customer_id, strftime(order_date, '%Y-%m-%d'), status,
sales_rep_id))
    FROM raw.orders""").fetchall())
print("inserted:", sorted(set(source_hashes) - set(wh_hashes)))
print("changed :", sorted(k for k in set(source_hashes) & set(wh_hashes) if source_hashes[k] !=
wh_hashes[k]))
```

```
inserted: [10178]
changed : [10176]
```

The hash comparison, which scanned every order, found the same two changes to `orders` that CDC reported from the log.

Watch out: an abandoned slot fills the disk. PostgreSQL keeps every log record a slot hasn't consumed. If a CDC reader stops for days and nobody notices, the log grows until the database server runs out of disk space, and the ERP stops. Monitor slot lag (Chapter 47), and drop slots you no longer use: `SELECT pg_drop_replication_slot('ch45_slot');`. The companion's `reset()` does this for you.

Choosing a strategy

If...	Consider
The table is small, or you're not sure	A full load
Rows are only ever added, never changed (events, log entries)	Incremental by ID or timestamp
Rows change, and a trustworthy <code>updated_at</code> exists	Incremental by timestamp , with a look-back window and an upsert
Rows change, no trustworthy column, table not huge	Hash comparison
Rows change, the table is large or busy, or you need deletes and low delay	Change data capture

Many platforms mix them: CDC for the big, busy order tables; full loads for products and employees; files and APIs for everything else.

45.7 Loading files reliably

Chapter 12 promised this chapter would cover loading large and messy files reliably. Files are still one of the most common ways data arrives, especially from spreadsheets, partners, banks, and older systems.

Bhiwandi Main's dispatch sheet for 2 January 2026 was saved from a spreadsheet. Here's the file as it appears in a text editor:

```
dispatch_id,order_id,dispatch_date,warehouse,qty_units,remarks
D-2601,10175,02-01-2026,Bhiwandi Main,100,"Delivered to site, signed"
D-2602,10173,02-01-2026,Bhiwandi Main,"1,200",Part load
D-2603,10172,02-01-2026,Bhiwandi Main,N/A,Awaiting count
D-2604,10171,02-01-2026,Bhiwandi Main,85,
D-2605,10170,02-01-2026,Bhiwandi Main,"2,040","Two trucks; second at 4 pm"
```

It looks tidy. It has five traps: an invisible **byte-order mark** (BOM) at the very start, which some tools read as part of the first column name; dates written day-first (`02-01-2026` is 2 January, not 1 February); quantities with thousands separators, which have to be quoted; a quantity of `N/A`; and Windows line endings with a blank line at the end.

What an automatic reader does

Modern readers try to detect everything for you. DuckDB's `read_csv` is good at it:

```
print(wh.execute("SELECT * FROM read_csv('exports/dispatch_2026-01-02.csv') LIMIT 3").fetchall())
print(wh.execute("DESCRIBE SELECT * FROM read_csv('exports/dispatch_2026-01-02.csv')").fetchall())
```

```
[('D-2601', 10175, datetime.date(2026, 1, 2), 'Bhiwandi Main', '100', 'Delivered to site, signed'),
 ('D-2602', 10173, datetime.date(2026, 1, 2), 'Bhiwandi Main', '1,200', 'Part load'), ('D-2603',
 10172, datetime.date(2026, 1, 2), 'Bhiwandi Main', 'N/A', 'Awaiting count')]
[('dispatch_id', 'VARCHAR', 'YES', None, None, None), ('order_id', 'BIGINT', 'YES', None, None,
None), ('dispatch_date', 'DATE', 'YES', None, None, None), ('warehouse', 'VARCHAR', 'YES', None,
None, None), ('qty_units', 'VARCHAR', 'YES', None, None, None), ('remarks', 'VARCHAR', 'YES', None,
None, None)]
```

Reading it. DuckDB handled the byte-order mark, the day-first dates, the quoted commas, and the blank line correctly. But look at the types: `qty_units` came in as **VARCHAR**, text, because of `"1,200"` and `N/A`. Nothing failed. A report that sums this column will either error much later or, in a spreadsheet, quietly skip the text values. Automatic detection is a helpful first look, and a risky way to run a daily load, because tomorrow's file may be guessed differently.

Watch out: guessed types change from file to file. If tomorrow's file happens to contain only plain numbers, the same column will be detected as an integer; the day after, as text again. Downstream code that worked yesterday breaks, or worse, silently changes behavior. In a scheduled load, declare the columns.

Declare the columns, keep the raw text, then type in staging

A reliable file load has two steps. First, load into raw with **declared columns**, keeping awkward fields as text so that nothing is lost or guessed. Second, convert to proper types in staging, and **set aside** the values that can't be converted.

```
wh.execute("""
CREATE OR REPLACE TABLE raw.dispatch AS
SELECT *
FROM read_csv('exports/dispatch_2026-01-02.csv',
    header = true,
    columns = {'dispatch_id': 'VARCHAR', 'order_id': 'INTEGER', 'dispatch_date': 'VARCHAR',
               'warehouse': 'VARCHAR', 'qty_units': 'VARCHAR', 'remarks': 'VARCHAR'})""")
print(wh.execute("SELECT dispatch_id, dispatch_date, qty_units, remarks FROM
raw.dispatch").fetchall())
```

```
[('D-2601', '02-01-2026', '100', 'Delivered to site, signed'), ('D-2602', '02-01-2026', '1,200',
'Part load'), ('D-2603', '02-01-2026', 'N/A', 'Awaiting count'), ('D-2604', '02-01-2026', '85',
None), ('D-2605', '02-01-2026', '2,040', 'Two trucks; second at 4 pm')]
```

The raw table holds exactly what the warehouse team wrote, including `N/A`. Now convert in staging:

```

wh.execute("CREATE SCHEMA IF NOT EXISTS staging")
wh.execute("""
    CREATE OR REPLACE TABLE staging.dispatch AS
    SELECT dispatch_id,
           order_id,
           strptime(dispatch_date, '%d-%m-%Y')::DATE           AS dispatch_date,
           warehouse,
           TRY_CAST(replace(qty_units, ',', '')) AS INTEGER) AS qty_units,
           remarks
    FROM raw.dispatch""")
bad = wh.execute("""
    SELECT dispatch_id, qty_units
    FROM raw.dispatch
    WHERE TRY_CAST(replace(qty_units, ',', '')) AS INTEGER) IS NULL""").fetchall()
print(wh.execute("SELECT dispatch_id, dispatch_date, qty_units FROM staging.dispatch ORDER BY
dispatch_id").fetchall())
print("rows needing attention:", bad)

```

```

[('D-2601', datetime.date(2026, 1, 2), 100), ('D-2602', datetime.date(2026, 1, 2), 1200), ('D-
2603', datetime.date(2026, 1, 2), None), ('D-2604', datetime.date(2026, 1, 2), 85), ('D-2605',
datetime.date(2026, 1, 2), 2040)]
rows needing attention: [('D-2603', 'N/A')]

```

How it works, line by line.

- `strptime(dispatch_date, '%d-%m-%Y')` reads the date with an **explicit format**: day, month, year. If a date doesn't match, the load fails loudly instead of swapping day and month.
- `replace(qty_units, ',', '')` removes the thousands separators, so `"1,200"` becomes `1200`.
- `TRY_CAST(... AS INTEGER)` converts to a whole number, and returns `NULL` instead of an error when it can't. A plain `CAST` would stop the whole load on `N/A`.
- The second query lists the rows where conversion failed, so a person can follow up.

Reconcile the file

Now prove the load is complete: compare what's in the file with what's in the tables.

```

with open("exports/dispatch_2026-01-02.csv", encoding="utf-8-sig") as f:
    data_lines = [l for l in f.read().splitlines()[1:] if l.strip()]
loaded = wh.execute("SELECT COUNT(*) FROM staging.dispatch").fetchone()[0]
usable = wh.execute("SELECT COUNT(*) FROM staging.dispatch WHERE qty_units IS NOT NULL").fetchone()[0]
print("data lines in file :", len(data_lines))
print("rows loaded          :", loaded)
print("rows with a usable quantity:", usable)
print("total units (usable rows):", wh.execute("SELECT SUM(qty_units) FROM
staging.dispatch").fetchone()[0])

```

```

data lines in file : 5
rows loaded          : 5
rows with a usable quantity: 4
total units (usable rows): 3425

```

What to tell the dispatch supervisor. All five dispatches were loaded. Four have quantities, totaling 3,425 units. Dispatch D-2603 for order 10172 has no quantity ("N/A, awaiting count"), so it's excluded from unit totals until the count arrives. Please send the corrected quantity, or update the sheet before tonight's load.

Simplification note. Counting lines works here because no field contains a line break. Real files sometimes do (an address or a remark typed over two lines, inside quotes), so production reconciliation counts *records* with a proper CSV parser, not raw lines, and compares against a count or total supplied by the sender when one exists.

A robust file-loading checklist

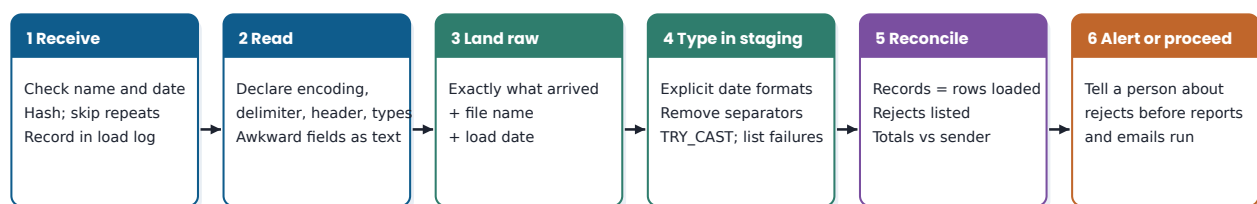


Figure 45.4 — A reliable file load, step by step. Every step either proves something or records something.

1. **Receive:** check the file name matches the expected pattern and date; hash it; skip exact repeats; log it.
2. **Read with declared settings:** encoding, delimiter, quote character, header, and column names and types. Don't rely on guessing in a scheduled load.
3. **Land raw:** keep the original values, plus the file name and load date on every row, so any number can be traced back to its file.
4. **Type in staging:** explicit date formats, clean separators, `TRY_CAST`, and a list of values that failed.
5. **Reconcile:** records in the file against rows loaded; rejected rows listed; totals checked against any control total the sender provides.
6. **Alert or proceed:** if anything was rejected or doesn't reconcile, tell a person before reports and emails run. Chapter 46 wires this into a pipeline.

Parquet: when files are designed for data

Chapter 2 introduced **Parquet**, a columnar file format that stores types, compresses well, and reads fast. Once a file has been cleaned, writing it as Parquet keeps its types for every later reader:

```
wh.execute("COPY staging.dispatch TO 'warehouse/dispatch_2026-01-02.parquet' (FORMAT parquet)")
print(wh.execute("DESCRIBE SELECT * FROM 'warehouse/dispatch_2026-01-02.parquet'").fetchall())
print("bytes: csv =", os.path.getsize("exports/dispatch_2026-01-02.csv"),
      " parquet =", os.path.getsize("warehouse/dispatch_2026-01-02.parquet"))
```

```
[('dispatch_id', 'VARCHAR', 'YES', None, None, None), ('order_id', 'INTEGER', 'YES', None, None, None), ('dispatch_date', 'DATE', 'YES', None, None, None), ('warehouse', 'VARCHAR', 'YES', None, None, None), ('qty_units', 'INTEGER', 'YES', None, None, None), ('remarks', 'VARCHAR', 'YES', None, None, None)]
bytes: csv = 374  parquet = 1208
```

Reading it. The Parquet file carries its types with it: `dispatch_date` is a `DATE` and `qty_units` an `INTEGER`, so no later reader has to guess. For this five-row file, Parquet is *larger* than the CSV, because it stores metadata and structure that only pays off with more data. Chapter 2 measured the opposite result on large files, where Parquet was far smaller and faster. Choose Parquet for data at scale and for types; don't expect it to shrink a tiny file.

Dialect note: other tools. The same ideas apply in other tools, with different syntax. In pandas, pass `dtype=`, `parse_dates=` with `dayfirst=True`, and `thousands=`"," to `read_csv`. In PostgreSQL, `COPY ... FROM ... WITH (FORMAT csv, HEADER true)` loads into a table whose types you've declared, and rejects the whole file on the first bad value, so load into a text-typed raw table first.

45.8 When the source changes shape

A **schema** is the structure of a dataset: its column names, types, and order. A **schema change** (or **schema drift**) is when a source changes that structure: a column is added, removed, renamed, or changes type.

On 5 January, Bhiwandi Main started recording which truck carried each dispatch, and someone tidied up a column name. The new file begins:

```
dispatch_id,order_id,dispatch_date,warehouse,units,remarks,vehicle_no
```

Compare its columns with what the load expects, before loading anything:

```
EXPECTED = ["dispatch_id", "order_id", "dispatch_date", "warehouse", "qty_units", "remarks"]
new_cols = [r[0] for r in wh.execute(
    "DESCRIBE SELECT * FROM read_csv('exports/dispatch_2026-01-05.csv', all_varchar =
true)").fetchall()]
print("missing columns:", [c for c in EXPECTED if c not in new_cols])
print("new columns      :", [c for c in new_cols if c not in EXPECTED])
```

```
missing columns: ['qty_units']
new columns      : ['units', 'vehicle_no']
```

Reading it. One expected column is missing (`qty_units`), and two new ones appeared (`units` and `vehicle_no`). A load that matched columns by *position* would still run, and would put `remarks` values where it expected quantities. A load that matched by name would find no quantities at all. Either way, the unit totals would be wrong.

How to handle schema changes

Different changes deserve different responses.

Change	Usually safe?	Sensible response
New column added	Yes, if you match columns by name	Load known columns; log the new one; add it to raw when someone confirms what it means
Column removed	No	Stop the load and alert; downstream reports depend on it
Column renamed	No, because it looks like one removed and one added	Stop and alert; map the new name only after confirming it means the same thing
Type changed (number becomes text)	No	Raw keeps text, so it loads; staging's rejected-values check will catch it; alert
Column order changed	Yes, if you match by name	Always match by name, never by position

Here, the right response is to **stop**, tell the warehouse team and the pipeline's owner, confirm that `units` means the same as `qty_units`, add the mapping, and add `vehicle_no` to the raw table. That conversation is part of a **data contract**: an agreement with a source's owner about what they'll send and how they'll warn you before changing it. Chapter 47 covers data contracts in practice.

Real-life example: the tidy-up that cost a week. A common story in data teams: someone renames a spreadsheet column to make it clearer, a nightly load keeps "succeeding", and a dashboard shows zero sales for a region for several days before anyone asks why. The fix takes minutes. Finding it takes days, because nothing failed. A column check like the one above turns that into an alert on the first night.

45.9 Pulling data from APIs

Chapter 2 introduced APIs and Chapter 18 called one from Python. Ingesting from an API adds four practical problems: **authentication**, **pagination**, **rate limits**, and **transient failures**.

Riverstone's CRM exposes its leads through a REST API. The companion's `mock_crm_api.py` behaves like a typical one: it needs a token, returns at most 20 leads per request, and, like real APIs under load, sometimes answers "too many requests".

```

from mock_crm_api import start_server
server = start_server()
BASE = "http://127.0.0.1:8045/api/leads"
HEADERS = {"Authorization": "Bearer practice-token-45"}

def get_with_retry(url, params, max_attempts=4):
    for attempt in range(1, max_attempts + 1):
        resp = requests.get(url, params=params, headers=HEADERS, timeout=10)
        if resp.status_code == 429:
            wait = int(resp.headers.get("Retry-After", "1"))
            print(f" page {params['page']}: 429 rate limited, waiting {wait}s (attempt
{attempt})")
            time.sleep(wait)
            continue
        resp.raise_for_status()
        return resp.json()
    raise RuntimeError(f"gave up after {max_attempts} attempts")

leads, page = [], 1
while page is not None:
    body = get_with_retry(BASE, {"page": page, "page_size": 20})
    print(f" page {page}: {len(body['data'])} leads")
    leads.extend(body["data"])
    page = body["next_page"]
print("leads fetched:", len(leads))

```

```

page 1: 20 leads
page 2: 429 rate limited, waiting 1s (attempt 1)
page 2: 20 leads
page 3: 3 leads
leads fetched: 43

```

How it works, line by line.

- `start_server()` starts the practice API in the background. Against a real CRM, you'd skip this and use its address.
- `HEADERS` carries the **token** that proves who you are. Real tokens are secrets: never write them into code or commit them to Git. Read them from an environment variable or a secrets manager (Chapters 52 and 64 cover secrets in production). Chapter 29 built a reusable API client with the same retry and pagination logic.
- `get_with_retry()` makes one request. `timeout=10` stops it waiting forever. If the answer is **429 Too Many Requests**, it reads the `Retry-After` header, waits that many seconds, and tries again, up to four times. `raise_for_status()` turns any other error (401, 404, 500) into an exception, so failures are never mistaken for empty data.
- The `while` loop follows **pagination**: it asks for page 1, then whatever `next_page` the API returns, until `next_page` is empty.

Reading it. The API returned 20, 20, and 3 leads. Page 2 was rate limited once; the code waited and retried, and nothing was lost. That's 43 leads in total.

What happens without a token

```

resp = requests.get(BASE, params={"page": 1})
print(resp.status_code, resp.json())

```

```
401 {'error': 'missing or invalid token'}
```

A **401 Unauthorized** response means the token is missing, wrong, or expired. Retrying won't help, which is why the retry function only retries 429s. A good rule: **retry what might succeed next time** (429, 503, timeouts), and **fail fast on what won't** (400, 401, 403, 404).

Load and reconcile

```
wh.execute("""
    CREATE OR REPLACE TABLE raw.crm_leads (
        lead_id      INTEGER PRIMARY KEY,
        created_at    TIMESTAMP,
        company_name  VARCHAR,
        email         VARCHAR,
        source        VARCHAR,
        owner_id      INTEGER,
        loaded_for    DATE) """)
wh.executemany("INSERT INTO raw.crm_leads VALUES (?, ?, ?, ?, ?, ?, DATE '2026-01-05')",
               [(l["lead_id"], l["created_at"], l["company_name"], l["email"], l["source"],
                  l["owner_id"])
                 for l in leads])
src_count = fetch_rows("SELECT COUNT(*) FROM leads")[1][0][0]
wh_count = wh.execute("SELECT COUNT(*) FROM raw.crm_leads").fetchone()[0]
print("source:", src_count, " warehouse:", wh_count, " match:", src_count == wh_count)
server.shutdown(); conn.close()
```

```
source: 43  warehouse: 43  match: True
```

All 43 leads arrived. Against a real CRM you usually can't count the source directly; instead, compare with the total the API reports (many APIs return one), or with a count exported from the CRM's own screen.

Watch out: offset pagination and moving data. This API pages by position ("records 21–40"). If a new lead is added while you're reading, every later record shifts by one, and you can skip or repeat a record. For data that changes during the read, prefer APIs that page with a **cursor** (a token pointing at the next record) or filter by a stable key or timestamp, and always reconcile the final count.

API ingestion checklist

- Keep **credentials** out of code; rotate them; use the least access the load needs.
- Always set **timeouts**.
- **Retry** 429, 503, and timeouts with increasing waits, respecting `Retry-After`; fail fast on 4xx errors.
- Follow **pagination** to the end, and prefer cursor pagination for changing data.
- Ask for **only what changed** when the API supports it (for example `updated_since=`), with a look-back window.
- Land the **raw response** (or its fields) with a load date, so you can re-process without calling the API again.

- **Reconcile** against a total from the source.
- Respect the provider's **limits and terms**. Some APIs charge per call or suspend keys that exceed limits.

45.10 Build or buy: connectors versus custom code

Everything in this chapter so far was custom code. In practice, many companies don't write ingestion for common tools at all. They use **managed connectors**: services and open-source tools that already know how to extract data from hundreds of popular databases and SaaS products, handle pagination, schema changes, and CDC, and load into a warehouse. Well-known examples include Fivetran and Airbyte, and cloud providers offer their own.

Question	Leans toward a managed connector	Leans toward custom code
Is the source a popular product with a maintained connector?	Yes	No, or the connector lacks what you need
How much engineering time do you have?	Little	Enough to build <i>and</i> maintain
How much data, and how does the tool charge?	Moderate volumes where pricing is affordable	Very high volumes where per-row pricing becomes expensive
How unusual is the source?	Standard APIs and databases	Custom files, older systems, internal tools, odd formats
Where can the data go?	Allowed to pass through a vendor's service	Must stay inside your network or country
Who fixes it when it breaks?	The vendor, for connector bugs	You

The honest summary: **buying saves building, not owning**. A connector still needs someone to choose what to sync, watch costs, reconcile results, and respond when a source changes. And custom code is rarely "free": the first version takes days; keeping it working as sources change takes years.

For Riverstone, a reasonable split would be a managed connector for the CRM if a maintained one exists for its product, CDC or scheduled queries for the ERP, and custom loading for the warehouse's dispatch sheet, which no vendor will ever have a connector for.

Interview extra point. When asked "how would you ingest data from X?", don't start with a tool. Ask three questions first: how big is it, how does it change (append-only, updates, deletes), and how fresh does it need to be? Then choose full, incremental, hash, or CDC, and say how you'd reconcile it. Chapter 77 has data engineering system design questions that use exactly this structure.

45.11 Web data: legal and ethical limits

Sometimes the data a business wants isn't in any of its systems: competitors' published prices, public tender notices, product reviews. Collecting data from websites automatically is **web scraping**. It can be legitimate, and it has real limits.

Before collecting any web data, check these, in order.

1. **Is there an official way?** An API, a data download, or a licensed data feed is almost always better: more reliable, and clearly permitted.
2. **What do the site's terms of use say?** Many sites prohibit automated collection. Breaking terms of use can have legal consequences, and it can get your company's network blocked.
3. **What does `robots.txt` say?** Most sites publish a file at `/robots.txt` saying which parts automated tools may visit. Respecting it is the minimum standard of good behavior.
4. **Does the data include personal information?** Names, phone numbers, email addresses, and profile details are personal data. Data protection laws, including India's Digital Personal Data Protection Act, 2023, and the EU's GDPR, apply to collecting and using personal data even when it's publicly visible. Treat personal data as off-limits unless your legal or compliance team has confirmed a lawful basis.
5. **Is the content protected by copyright?** Reviews, articles, images, and databases can be. Collecting facts is different from copying creative content.
6. **Will your collection harm the site?** Send requests slowly, identify your tool truthfully, and never try to get around logins, paywalls, or blocking measures.

When in doubt, ask your company's legal or compliance team before collecting anything. This section is general guidance, not legal advice; laws differ between countries and change over time. Chapter 64 covers privacy and data governance in depth.

Watch out: "it's public" is not permission. Being visible in a browser doesn't make data free to collect, store, and reuse. The questions above apply to public pages too.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Judging a load by whether it finished	Dashboards drift from the source; nobody knows since when	Reconcile counts and totals against the source after every load (45.3)
Incremental load by ID on rows that get updated	Status counts differ from the source; cancelled orders still open (45.4)	Use <code>updated_at</code> with a look-back, hash comparison, or CDC
Trusting <code>updated_at</code> without checking it	Occasional missing changes, especially after bulk fixes	Test it against a hash comparison before relying on it
Forgetting deletes	Rows deleted in the source live forever in the warehouse	Full load, hash comparison, or CDC; or a source "deleted" flag

Mistake	Symptom	Fix
Plain <code>INSERT</code> in a re-runnable load	Duplicate rows after a retry	Primary keys in raw, and upserts (45.5)
Building hash strings differently on each side	Every row looks changed	Same columns, order, separator, and date format on both sides
Leaving a CDC slot unconsumed	Source database disk fills; the ERP stops	Monitor slot lag; drop unused slots (45.6)
Letting a reader guess column types in a daily load	Columns switch between number and text from day to day	Declare columns; type in staging (45.7)
Parsing dates without an explicit format	2 January loaded as 1 February	<code>strptime</code> with the source's exact format
<code>CAST</code> on messy values	One bad value stops the whole load	<code>TRY_CAST</code> , and list the failures
Matching file columns by position	Values land in the wrong columns after a change	Match by name; check expected columns first (45.8)
Retrying every error	Endless retries on a bad token; accounts locked	Retry 429, 503, timeouts; fail fast on 4xx
No timeout on API calls	A load hangs all night	Always set a timeout
Tokens in code	A secret leaks through Git	Environment variables or a secrets manager
Scraping without checking terms or personal data	Legal risk, blocked networks	Official sources first; check terms, robots.txt, privacy law (45.11)

In the real world: the orders that never cancelled

Riverstone's first proper data pipeline went live in February, built by a data engineer the company had hired on contract, with Meera as the business owner. Every night it loaded new orders from the ERP into the warehouse, and every morning the Daily Sales Flash came from the warehouse instead of from Meera's spreadsheet. It was fast, it never failed, and for three weeks everyone was pleased.

Then Vikram brought a printout to the Monday sales review. "The Flash says we have eleven open orders for Hotel Sai Palace. Their buyer told me on Friday they cancelled six of those in January."

Meera opened the ERP. The six orders were marked Cancelled. She opened the warehouse. The same six orders said Pending.

She'd read enough about ingestion to guess the cause before she found it. The pipeline loaded orders with an ID higher than the last one it had seen. Cancelling an order changes its status, not its ID. Every cancellation, every delivery, every correction made to an existing order after its first night had never reached the warehouse.

"How many orders are wrong?" Anita asked.

"I don't know yet," Meera said. "That's the worse problem. Nothing told us."

That afternoon, the engineer and Meera did three things. First, they measured the damage: a hash comparison across every order since the pipeline started. It found 214 orders whose status in the warehouse didn't match the ERP. Second, they fixed the data, with an upsert of every changed row, and re-ran the Flash for the affected days, so the sales team had correct history. Third, they fixed the design. The ERP had no reliable last-updated column, so the engineer set up a nightly hash comparison over the last 90 days of orders, plus a full comparison every Sunday, and added a reconciliation check: order counts by status in the warehouse must match the ERP before the Flash is sent.

The check failed once more, two weeks later, when someone corrected a batch of orders from the previous quarter, outside the 90-day window. This time nothing reached the sales team. The Flash was held, an alert went to Meera at 6:10 a.m., the Sunday full comparison was run early, and the Flash went out at 7:30 with the right numbers.

At the monthly review, Anita asked the question Meera had been expecting. "Why didn't the pipeline do this from the start?"

"Because it looked like it worked," Meera said. "It loaded exactly what it was built to load. We built it to load the wrong thing, and never checked it against the ERP."

What made this work.

- **Meera recognized the pattern:** a load that never fails, with numbers that drift, points to what the load isn't reading.
- **They measured before fixing:** the hash comparison showed the full size of the problem, not only the six orders someone happened to notice.
- **They fixed data, design, and detection:** corrected history, a change-detection method that catches updates, and a reconciliation check that stops bad numbers before they're sent.
- **The check proved its value** the next time something changed outside the plan.

Tools

- **PostgreSQL 16**, with `wal_level = logical` for section 45.6. On a company database, ask the administrator; don't change it yourself.
- **Python 3.12** with `psycopg2` (PostgreSQL driver), `duckdb` (local analytical database), and `requests` (HTTP). Install with `pip install psycopg2-binary duckdb requests`.
- **The Chapter 45 companion folder** (`companion/ch45/`): `reset_ch45.py`, `apply_day.py`, `mock_crm_api.py`, `make_files.py`. Run Python from inside that folder. If your PostgreSQL needs a user or password, set the environment variable `RIVERSTONE_SOURCE`, for example `dbname=riverstone_source user=postgres password=yourpassword`, and use the same string for `SRC`.
- **Versions used for the outputs shown:** Python 3.12.3, DuckDB 1.5.5, psycopg2 2.9.13, requests 2.33.1, PostgreSQL 16.

- **Worth knowing about, not needed here:** Debezium for production CDC; managed connectors such as Fivetran and Airbyte; `dlt` and Singer-style tools for writing connectors in Python.
-

The project: incremental ingestion for Riverstone

Goal: a small, re-runnable ingestion job that loads Riverstone orders, order lines, CRM leads, and dispatch files into the warehouse, detects changes correctly, and proves every load with a reconciliation report. Chapter 46 turns it into an orchestrated pipeline, and Chapter 47 adds quality checks and freshness alerts.

Option A: Riverstone. Use the companion environment.

Option B: your own data. Use a database, API, or regular file you have permission to read. Don't use production systems for practice without your administrator's agreement, and keep credentials out of anything you share.

Steps

1. **Plan the sources.** For each of `orders`, `order_items`, `customers`, `products`, CRM leads, and dispatch files, write down: source type, system of record, size, whether rows are updated or deleted, and your chosen strategy (full, incremental, hash, CDC) with one sentence of reasoning.
2. **Create the raw tables** in the warehouse with declared types and primary keys, plus `loaded_at_run` (a run ID) on every table.
3. **Build a load log table:** run ID, source, strategy, rows read, rows inserted, rows updated, rows rejected, reconciliation result.
4. **Full loads** for `customers` and `products`.
5. **Change detection for `orders` and `order_items`**, using hash comparison or CDC, applied with upserts.
6. **The CRM leads API**, with authentication, pagination, retries on 429, and a final count check.
7. **Dispatch files**, with file hashing, declared columns, typed staging, a rejected-rows list, and a column check that stops the load on missing or renamed columns.
8. **A reconciliation report**, printed at the end of every run: for each source, rows in the source (or file), rows in the warehouse, and order counts by status. Mark any mismatch clearly.
9. **Test it:** run `reset()`, run your job, `apply_day(1)`, run it again, `apply_day(2)`, run it again, then run it a fourth time with no changes. The reconciliation report must match the source every time, and the fourth run must change nothing.
10. **Break it on purpose:** load `dispatch_2026-01-05.csv` and confirm your column check stops it with a clear message.

Stretch goals

- Replace the hash comparison for orders with the CDC slot, applying each change with an upsert.
- Add a deleted order to `apply_day` (in your own copy) and show which strategies notice it.

- Add a look-back window to an `updated_at` -style incremental load of lead stage history, and show it handles a late change.
-

You've got it when...

- ☐ You can name the main kinds of source system and one typical difficulty of each.
 - ☐ You can explain raw, staging, and modeled layers, and why raw keeps data as it arrived.
 - ☐ You can write a full load with declared types and reconcile it by count and by category.
 - ☐ You can explain, and demonstrate, why an incremental load by ID misses updates and deletes.
 - ☐ You can list three ways an `updated_at` watermark can still miss changes.
 - ☐ You can detect inserted, changed, and deleted rows with row hashes, and explain why both sides must build the string identically.
 - ☐ You can explain idempotency and show that an upsert can be safely re-run.
 - ☐ You can skip already-loaded files with a file hash, and say when that would be wrong.
 - ☐ You can explain change data capture, replication slots, and the risk of an abandoned slot.
 - ☐ You can choose between full, incremental, hash, and CDC for a given table, with reasons.
 - ☐ You can load a messy CSV with declared columns, explicit date formats, `TRY_CAST`, and a rejected-rows list, and reconcile it against the file.
 - ☐ You can detect a schema change before loading and choose the right response.
 - ☐ You can ingest from a paginated, authenticated, rate-limited API with retries and timeouts.
 - ☐ You can argue build versus buy for a given source.
 - ☐ You can list the checks to make before collecting web data.
-

Recap

- **Ingestion** moves data from source systems into the warehouse. Its dangerous failures are loads that succeed with wrong data.
- Sources include **databases**, **files**, **APIs**, **events**, and **SaaS tools**; the **system of record** is the source that's officially correct.
- Data lands in a **raw** layer as it arrived, then a typed **staging** layer, then **modeled** tables.
- A **full load** replaces everything: simple, self-correcting, and sees updates and deletes. It stops being practical for large or busy tables.
- An **incremental load** uses a **watermark**. By ID it misses **updates** and **deletes**; `updated_at` helps, with a **look-back** window, if it's trustworthy.
- **Row hashes** detect inserts, updates, and deletes without help from the source, at the cost of scanning keys. **File hashes** skip content already loaded.
- **Upserts** make loads **idempotent**: safe to run again.

- **Change data capture** reads the database's **write-ahead log** through a **replication slot**: every change, in order, without scanning. Unconsumed slots fill disks.
- **Reliable file loads** declare columns, keep raw text, type in staging with explicit formats and `TRY_CAST`, list rejects, and **reconcile** against the file.
- **Schema changes** should be detected before loading; match columns **by name**, and stop on removed or renamed columns.
- **API ingestion** needs secure **authentication**, **timeouts**, **pagination**, and **retries** for 429 and 503 only.
- **Managed connectors** save building, not owning.
- **Web data** needs an official source where possible, and checks on terms, robots.txt, personal data, and copyright.

Practice exercises

Run `reset()` before each exercise that uses the companion environment, unless it says otherwise.

Warm-up

1. For each Riverstone table or source, choose a loading strategy (full, incremental by ID, incremental by timestamp, hash comparison, or CDC) and give one reason: (a) `products`; (b) website enquiry events that are only ever added; (c) `orders`, which are updated but have no `updated_at`; (d) the CRM's lead stage history, where every stage change adds a row with a timestamp.
2. Which HTTP status codes should an ingestion job retry, and which should make it fail immediately? Classify: 200, 401, 404, 429, 500, 503, and a timeout.
3. Explain in two sentences why `concat_ws('|', ...)` uses a separator when building a row hash.

Core

4. After `reset()` and a full load of orders, run `apply_day(1)` and then `apply_day(2)` *without loading in between*. Using the hash comparison from section 45.5, which order IDs will be inserted and which changed? Explain why order 10176 appears in only one of the lists.
5. Write a function `load_orders_full()` that performs a full load of `orders` into `raw.orders` and returns the source and warehouse counts by status. Why does a full load not need a separate method for detecting updates?
6. A dispatch file has the line `D-2610,10180,13-01-2026,Bhiwandi Main,"3,150",.` What will `staging.dispatch` contain for `dispatch_date` and `qty_units`? What would happen if the date were parsed with the format `'%m-%d-%Y'`?
7. Using the reconciliation output in section 45.7, a manager asks: "What were total units dispatched on 2 January?" Write the answer you'd send, including what's missing.
8. Modify `get_with_retry()` so that it waits longer after each failed attempt (1, 2, 4, 8 seconds), unless the server sends a `Retry-After` header. Why is increasing the wait better than retrying immediately?

9. Explain why the plain `INSERT` in section 45.5 raised an error on the second run. What would have happened if `raw.orders` had no primary key?

Stretch

- l0. Extend the column check from section 45.8 into a function `check_columns(path, expected)` that returns `"ok"`, `"new columns only"` (safe to load known columns), or `"stop"` (a column is missing), and a list of the differences. Run it on both dispatch files.
- l1. Design a strategy for an `invoices` table with 80 million rows, where invoices are sometimes corrected up to 60 days after they're raised, rows are never deleted, and there's no trustworthy `updated_at`. The ERP team won't allow CDC. Describe what you'd load nightly and weekly, and how you'd reconcile.
- l2. The CRM API pages by offset, and new leads arrive while you read. Describe a sequence in which a lead is skipped, using page size 20, and propose two fixes.

Think about it (no code needed)

- l3. Riverstone is choosing between a managed connector for its CRM and custom code. List three questions you'd ask before deciding, and what answer would push you each way.
- l4. A marketing colleague wants to collect the names and phone numbers of shop owners listed on a public business directory, to call them about Riverstone's products. What would you tell them before anything is collected?

Key terms

ingestion · source system · system of record · raw layer · staging layer · modeled layer · DuckDB · full load · reconciliation check · incremental load · watermark · last-updated timestamp · look-back window · hash (row hash, file hash) · upsert · idempotent · load log · write-ahead log (WAL) · change data capture (CDC) · replication slot · byte-order mark (BOM) · declared columns · `TRY_CAST` · rejected rows · Parquet · schema · schema change (schema drift) · data contract · authentication token · pagination (offset, cursor) · rate limit (429) · `Retry-After` · timeout · managed connector · web scraping · `robots.txt` · personal data

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 46, Pipelines & Orchestration**, turns this chapter's loads into a scheduled, orchestrated pipeline, with idempotency tested by a deliberate failure, backfills, alerts, and the Daily Sales Flash sent only after its data passes checks.
- **Chapter 47, Data Quality, Observability & Contracts**, turns reconciliation checks into automated tests, adds freshness monitoring and slot-lag alerts, and puts data contracts like the one in section

45.8 into practice.

- **Chapter 49, Storage, Warehouses & Lakehouses**, explains how warehouses store raw and staging layers, and covers ACID transactions, partitioning, and cloud warehouse costs.
- **Chapter 50, Streaming & Real-Time**, takes change data capture from a daily read to a continuous stream.
- **Chapter 51, Data Activation**, reverses the direction: writing warehouse data back into the CRM and ERP through APIs, with the same care for idempotency, retries, and systems of record.
- **Chapter 64** covers privacy, security, and governance, including the rules behind section 45.11.
- **Part VIII:** data engineering and ingestion questions appear in Chapter 72, and system design cases in Chapter 77.

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) **Full load:** `products` is small and changes occasionally, so reloading it is cheap and always correct. (b) **Incremental by ID or timestamp:** rows are only ever added, so a watermark can't miss updates. (c) **Hash comparison** (or CDC if the ERP team allows it): rows are updated, and there's no trustworthy column to find the changes. (d) **Incremental by timestamp** on `entered_at`, with a look-back window: each change adds a new row with a timestamp. A full load is also reasonable for (d) at Riverstone's size.
2. Retry: **429** (rate limited), **503** (temporarily unavailable), and **timeouts**. **500** is borderline: retry a small number of times, because it's sometimes temporary, then fail. Fail immediately: **401** (authentication won't fix itself) and **404** (the address is wrong). **200** is success: no retry.
3. Without a separator, different rows can produce the same string: values `1` and `23` join to `123`, exactly like `12` and `3`, so a change could go unnoticed. A separator that doesn't appear in the data keeps each value's boundaries, so different rows give different strings and different hashes.
4. Inserted: **10176, 10177, 10178**. Changed: **10174, 10175**. Order 10176 was created on day 1 and changed on day 2, but the warehouse never saw its first version, so it counts as new: its latest version is inserted. Hash comparison sees the *current* difference between source and warehouse, not the history of changes in between; CDC would report both the insert and the update.
5. A sample answer:

```
def load_orders_full():
    cols, rows = fetch_rows("SELECT order_id, customer_id, order_date, status, sales_rep_id FROM
orders")
    wh.execute("DELETE FROM raw.orders")
    wh.executemany("INSERT INTO raw.orders VALUES (?, ?, ?, ?, ?)", rows)
    src = dict(fetch_rows("SELECT status, COUNT(*) FROM orders GROUP BY status")[1])
    tgt = dict(wh.execute("SELECT status, COUNT(*) FROM raw.orders GROUP BY status").fetchall())
    return src, tgt
```

A full load replaces every row with the source's current version, so updated rows arrive in their new state and deleted rows disappear. There's nothing to detect. For a production version, wrap the delete and insert in one transaction, so readers never see an empty table.

6. `dispatch_date = 2026-01-13` (13 January 2026), and `qty_units = 3150`. With `'%m-%d-%Y'`, the parser would read month 13, which doesn't exist, so the load would fail with an error. That failure is useful: with a day of 12 or less, the wrong format would not fail, and would silently swap day and month.

7. "Bhiwandi Main recorded five dispatches on 2 January. Four have quantities, totaling 3,425 units. The fifth, D-2603 for order 10172, has no quantity yet (marked 'awaiting count'), so the true total is 3,425 plus that dispatch. I've asked the warehouse for the count and will update the figure once it arrives." The key points are the total, what's excluded, and that it's being followed up.

8. A sample answer:

```
def get_with_retry(url, params, max_attempts=5):
    for attempt in range(1, max_attempts + 1):
        resp = requests.get(url, params=params, headers=HEADERS, timeout=10)
        if resp.status_code in (429, 503):
            wait = int(resp.headers.get("Retry-After", 2 * (attempt - 1)))
            time.sleep(wait)
            continue
        resp.raise_for_status()
        return resp.json()
    raise RuntimeError(f"gave up after {max_attempts} attempts")
```

The waits without a `Retry-After` header are 1, 2, 4, 8, and 16 seconds. This is **exponential backoff**. Retrying immediately hits a server that's already overloaded or limiting you, which usually fails again and can make things worse for everyone. Waiting longer each time gives the server room to recover. Production code often adds a small random amount (**jitter**) so many clients don't retry at the same moment.

9. `raw.orders` has `order_id` as its **primary key**, and orders 10176 and 10177 already existed, so inserting them again violated the key and DuckDB refused. Without a primary key, the insert would have **succeeded silently**, creating duplicate rows: order counts and revenue for those orders would be doubled in every report, with no error anywhere.

10. A sample answer:

```
def check_columns(path, expected):
    cols = [r[0] for r in wh.execute(
        f"DESCRIBE SELECT * FROM read_csv('{path}', all_varchar = true)".fetchall())
    missing = [c for c in expected if c not in cols]
    extra = [c for c in cols if c not in expected]
    if missing:
        return "stop", {"missing": missing, "new": extra}
    if extra:
        return "new columns only", {"new": extra}
    return "ok", {}
```

On `dispatch_2026-01-02.csv`, it returns `ok`. On `dispatch_2026-01-05.csv`, it returns `stop`, with `qty_units` missing and `units` and `vehicle_no` new. The rename is reported as one missing and one new column, which is why it must stop rather than guess.

11. One reasonable design: **Nightly**, compute row hashes for invoices raised in the **last 60 days** (the correction window) plus a small margin, say 70 days, compare with the warehouse, and upsert changes; load new invoices in the same step. **Weekly**, compare hashes for the whole table **in monthly chunks**: compute one hash per month (for example, a hash of the sorted row hashes) on both sides, and only compare row by row for months whose chunk hash differs. **Reconcile** every night: invoice count and total amount by month for the last 70 days, and for all months weekly. Run the heavy queries against a read replica or outside business hours, with the ERP team's agreement. Because rows are never deleted, delete detection can be weekly only. Other designs are acceptable if they cover the 60-day correction window, limit the load on the ERP, and reconcile.

12. Suppose there are 45 leads, sorted by `lead_id`. You read page 1 (records 1–20). Before you request page 2, a new lead is added that sorts **before** some you haven't read yet (for example, if the API sorts by newest first, it appears at position 1). Every record shifts one place later. Page 2 now returns records 21–40, which are what used to be records 20–39: you read the old record 20 twice. The same shift in the other direction (a record deleted, or a sort where the new record comes first) makes you **skip** a record. Fixes: (1) use **cursor pagination** or a stable filter such as `lead_id > last_seen_id`, which isn't affected by inserts elsewhere; (2) de-duplicate by `lead_id` on load and **reconcile** the final count against the API's total, re-reading if it doesn't match. Reading only records created or updated before a fixed cut-off time also helps.

13. Useful questions: (a) *Is there a maintained connector for our exact CRM product, covering the objects we need?* Yes pushes toward buying; no, or only partly, toward custom code. (b) *What will it cost at our data volume and sync frequency, and what does our engineering time cost?* Affordable pricing and little engineering time push toward buying; high per-row costs at large volumes, or spare engineering capacity, push toward building. (c) *Is our customer and lead data allowed to pass through a vendor's service, and where is it processed?* If policy or law requires it to stay inside our systems or country, that pushes toward custom code or a self-hosted tool. Other good questions: who fixes it when it breaks, how quickly it handles schema changes, and whether it supports incremental syncs.

14. Tell them to stop and check before collecting anything. **Names and phone numbers are personal data**, and data protection laws such as India's Digital Personal Data Protection Act, 2023, apply even when the directory is public; using them for sales calls needs a lawful basis, which the company's legal or compliance team must confirm. Check the directory's **terms of use**, which often prohibit automated collection and commercial reuse, and its `robots.txt`. Ask whether there's an **official route**: a paid listing service, a licensed data provider, or the directory's own advertising options. And note that India's rules on unsolicited commercial calls also apply. The safest recommendation is to involve legal or compliance first, and to prefer sources where shop owners have agreed to be contacted.